

ARC / CIRCLE 7-WEEK HACKATHON · AGENTIC ECONOMY TRACK · FINAL SUBMISSION

# ACR – the Arc Compute Rate

Machine commerce just got its **SOFR** – and it prints its own **attack cost**.

A manipulation-resistant reference rate for machine services, live on Arc testnet – and the cash-settled futures venue that trades on it.

[arc-compute-rate.vercel.app](https://arc-compute-rate.vercel.app) · [github.com/kaustubh76/ACR---Arc-Compute-Rate](https://github.com/kaustubh76/ACR---Arc-Compute-Rate)

## THE PROBLEM

# Agents trade machine services with **no trustworthy price**

- Agents already pay per call for inference, GPU time, and data – but there is **no benchmark** to settle contracts against or hedge with.
- The raw payment exhaust is **noisy, batched, and adversarial**.
- A naive volume-weighted average is an open invitation: our own red team moves it **+122%** with wash trades.

No benchmark means no hedging, no term markets, no credit. Machine commerce is stuck at spot.

# Meet the hedger: an agent business with a floating USDC burn

- **Portia**, an autonomous paralegal fleet, resells LLM work at fixed prices but pays for inference per call – **~100M tokens/mo × 0.49236 \$/1k tokens** (the ACR-INF print) **≈ \$49,236/mo**, all floating. One +10% month is **+\$4,924** of unbudgeted burn.
- The fix: go **long ACR-INF futures**. Cash-settled – at expiry the venue **freezes the freshest oracle print** ( $\leq 2$  h old, enforced on-chain) and pays the difference in USDC. **No delivery, no GPU repossession, no seller cooperation.**
- A hedge is only as good as its settlement print: every ACR print ships its **CI** and its **attack-cost-per-bp**, so both sides can read – on-chain – what bending the settle would cost.
- **This loop is live on Arc testnet:** `ACRFutures 0x29d97c62...82642fe` – **three books** (ACR-INF, ACR-GPU, ACR-DATA), 10× multiplier, 2000 bp initial margin on this deployment, socialized-loss clearing, settling against the same ACROracle.
- **And an agent already runs Portia's playbook.** It pays \$0.0001 for the print over x402, reads its own book, and trades the gap to its mandate – from a Circle agent wallet, unattended. **It**

# Every Circle wallet model, doing a job – not ticking a box

| AGENT                | WALLET                                     | PAYS FOR                          | DECIDES                                     |
|----------------------|--------------------------------------------|-----------------------------------|---------------------------------------------|
| <b>press</b>         | Developer-Controlled                       | gas to sign 3 prints/hr           | <b>nothing, deliberately</b>                |
| <b>maker · taker</b> | Developer-Controlled ×2                    | collateral, roll gas, hourly fill | size, against both sides' margin            |
| <b>hedger</b>        | <b>Agent wallet</b> (SCA trades, EOA pays) | the print, then its own margin    | everything – and refuses rather than clamps |
| <b>reader</b>        | <b>User-controlled</b> SCA, PIN            | nothing: Gas Station sponsors     | their own trade, their own exit             |
| <b>CLI buyer</b>     | raw EOA + <b>x402-batching</b>             | \$0.0001 × 13 resources           | which listings to buy                       |

**We consume Circle's Skills and publish one back** ( **skills/acr-hedge** ), and **make gateway-deposit** (Unified Balance Kit) makes the last operator step something an agent does for itself.

## THE PRODUCT

Three indices, live on Arc testnet – chain **5042002**

| INDEX           | MEASURES                 | LATEST ON-CHAIN VALUE |
|-----------------|--------------------------|-----------------------|
| <b>ACR-INF</b>  | Inference – \$/1k tokens | <b>0.49236</b>        |
| <b>ACR-GPU</b>  | GPU compute – \$/GPU-sec | <b>0.01110</b>        |
| <b>ACR-DATA</b> | Data egress – \$/MB      | <b>0.00209</b>        |

Every hourly print ships **three numbers**: the rate, a confidence interval, and the **attack-cost-per-bp** (ACR-INF  $\approx$  **0.0094 USDC/bp**).

Read live from ACROracle at final submission – posted **hourly, in-cloud, signed by a Circle custody wallet** – and now the print the **futures venue freezes at expiry**.

## THE HEADLINE RESULT

**\$8,000** wash attack. The attacker got  $\leq 2.39\%$ .

| INDEX    | NAIVE VWAP MOVED | ACR MOVED     | RESISTANCE  |
|----------|------------------|---------------|-------------|
| ACR-INF  | <b>+110.8%</b>   | <b>+0.20%</b> | <b>562x</b> |
| ACR-GPU  | <b>+107.1%</b>   | <b>-1.02%</b> | <b>105x</b> |
| ACR-DATA | <b>+122.5%</b>   | <b>+2.39%</b> | <b>51x</b>  |

36,000 adversarial authorizations; the attacker burned **\$147.60** in fees. Claims are **CI-gated** ( **make eval-gate** ) so they cannot silently drift.

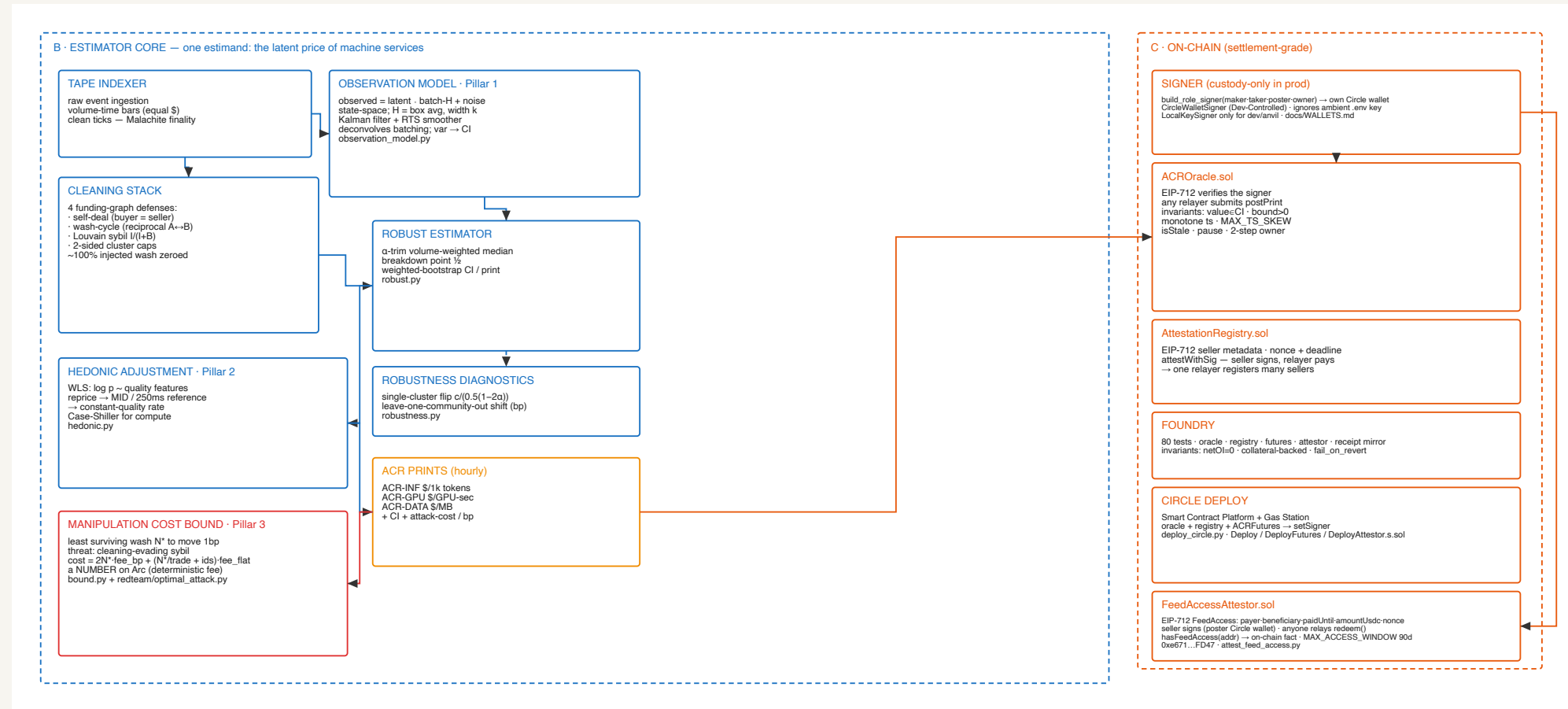
# Six **mathematical** dependencies – not deployment conveniences

| ARC PROPERTY                     | WHAT IT BUYS THE ESTIMATOR                             |
|----------------------------------|--------------------------------------------------------|
| Malachite deterministic finality | Clean tick timestamps                                  |
| Gateway: one canonical rail      | Complete observation, no selection bias                |
| x402 nanopayments                | Agents are the index's paying customers                |
| USDC numeraire                   | No gas-token deflator in the signal                    |
| <b>Deterministic USDC fees</b>   | The attack cost is <b>a number, not a distribution</b> |
| Agent Marketplace                | Native machine-to-machine discovery                    |

**And the bridge nobody else has to build:** Gateway settles nanopayments off-chain, so "this wallet paid for the feed" normally lives only in a seller's ledger. **FeedAccessAttestor** signs that receipt with the same custody wallet that signs prints, so **an off-chain payment becomes an on-chain right** – which is what lets a venue rebate fees to the wallets that paid for its index.

## ARCHITECTURE · ONE PIPELINE

# Adversarial exhaust in → settleable print out



Tape → clean (wash · sybil) → deconvolve (Kalman/RTS) →  $\alpha$ -trim median + hedonic → print + CI + attack-cost → EIP-712 oracle → futures settle → agents buy the rate back via x402.

# Recovering the latent constant-quality price

| PILLAR                       | WHAT IT DOES                                                                                                                      |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Observation model</b>     | Kalman + RTS smoother deconvolves Gateway's settlement batching                                                                   |
| <b>Cleaning</b>              | 4 funding-graph defenses (self-deals, wash cycles, sybil clusters, caps) – ~100% of injected wash zeroed                          |
| <b>Robust core + hedonic</b> | Trimmed volume-weighted median (breakdown point $\frac{1}{2}$ ) + bootstrap CI; quality-adjusted via on-chain seller attestations |
| <b>Manipulation bound</b>    | Prices the cheapest surviving attack per bp – validated attainable by <code>redteam/optimal_attack.py</code>                      |

Full spec: `docs/methodology.md` – published before liquidity, the way SOFR was.

# Deployed, printing, hedgeable – all seven weeks shipped

| CONTRACT / SURFACE                               | LIVE PROOF                                                                                           |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>ACROracle</b> 0x4f00e3BD...9092609            | Hourly EIP-712 prints, Circle custody signer                                                         |
| <b>AttestationRegistry</b> 0x23ae3E1A...2f63dFb7 | 4 seller attestations on-chain                                                                       |
| <b>ACRFutures</b> 0x29d97c62...82642fe           | Three live books (INF/GPU/DATA), <b>maker is a Circle custody wallet</b> , cash-settles on the print |
| <b>Terminal + Seller API</b>                     | API x402-gated at <b>\$0.0001/query</b> , fail-closed                                                |

make ci green 2026-09-07: **555 py** · **156 forge** (incl. futures + invariants) · **106 terminal** · **13/13 agent** · resistance **4/4** · interop **12/12** · glossary **426/426** · GitHub CI **6/6**.

Real settlement through **Circle Gateway** – batch-UUID receipts in-repo and on the public /exchange tape; Circle's own CLI paid the gate (payable).

Provenance labeled on every value – sim / gateway-ref / tx. Reproduce: make setup && make ci && make demo.

# Attack the index, then hedge on it

1. `/` – the print ticking live: rate + CI + attack cost. New to the jargon? Flip the masthead to **plain** – the whole paper re-sets in plain English.
2. `/attack` – press the button: wash flow floods in, naive VWAP swings **~+110%**, **ACR holds**, and the counter burns the attacker's USDC. Offline twin: `make demo` – one command, ~1 minute.
3. `/curve` – the term structure, the **live futures desk** read from ACRFutures on-chain, and **the Public Desk: trade it yourself**. Open a Circle user-controlled wallet in the browser (your PIN, our gas), take the stake, place a real fill, withdraw it again – the key never leaves your device, so every step is a challenge only your PIN can sign.
4. `/exchange` – **the hedger, both legs in one card, which is what this slide's title promised**.
  - prints it bought: real **Circle Gateway batch UUIDs**, \$0.0001 each, settled by the agent's own wallet – a settlement reference, not a hash, because Gateway batches off-chain. Then the joint: the venue fills every trade at that same print, so the card prints the arithmetic live – **2.47 × 0.49236 × 10 = 12.16 USDC** was the position those prints bought when this slide was cut.
  - fills it took is the on-chain half. An agent at its mandate stops trading, so when its fills have aged

# Arc mainnet is September 16. This is day-one infrastructure.

- **Nothing here is a testnet exhibit.** The estimator, the oracle, the venue and the gate are chain-first; the only Arc-specific assumptions are the ones that make the manipulation bound a number at all – deterministic USDC fees, USDC as gas, sub-second finality.
- **The one thing we are missing, named:** a dependent that is not us. The venue settling on the feed is a real dependent – but we deployed both. One outside consumer turns "our two components agree" into "someone else is exposed to this number." The Marketplace listing is submitted; Circle's Discovery API carries no Arc network yet.
- **And a community, with something to hold.** Post-hackathon: **\$GPOOR** on **ACTFUN**, Arc's own community-mined launchpad – you mine it by complaining about compute prices on-chain, with the live ACR print in the complaint. It belongs after a submission, not inside one: the mining window is an hour, and a launch nobody can mine is not a fair launch.

**Stop whining. Hedge.**

WHY THIS WINS

# The rate machine commerce settles on

- **Judge fit:** ICE – a benchmark administrator – is on Arc's testnet roster; Apollo, BNY, Mastercard are rate-native.
- **The neutrality moat:** Circle can't own the benchmark (the LIBOR lesson) – ACR is a partner, not a feature.
- **The loop closes:** resistant print → on-chain oracle → **cash-settled futures** + x402-paying machine customers → attestations → a better print.
- The first machine-commerce benchmark with a **live derivative venue** – three books, a term structure, and agents on both sides of it.

**Terminal** `arc-compute-rate.vercel.app` · **API** `acr-api-1fto.onrender.com`

**Contracts** `0x4f00e3BD...` / `0x23ae3E1A...` / `0x29d97c62...` on `testnet.arcscan.app` ·  
`docs/SUBMISSION.md`